CONTROL INSTRUCTIONS

CONTROL INSTRUCTIONS..... 1
Introduction..... 2
Types of Control Instructions..... 2
Decision Control Instructions..... 3
(a) if Statement..... 3
(b) if-else..... 11
(c) else-if ladder..... 17
(d) Nested if-else in C..... 24
(e) switch Statement..... 28
Example 1: Display Day of Week..... 33
Example 2: Simple Calculator..... 34
Example 3: Check Vowel or Consonant..... 35
Example 4: Menu Driven Program..... 36
Example 5: Even or Odd Using switch..... 37
Example 6: Arithmetic Operations..... 37
Loop Control Instructions..... 38
1) for Loop in C..... 38
Example 1: Print Numbers from 1 to 10..... 43
Example 2: Find Sum of First 10 Natural Numbers..... 44
Nested for Loop..... 46
2) while Loop..... 47
Example 1: Print Numbers from 1 to 10..... 49
Example 2: Sum of First 10 Natural Numbers..... 50
for Loop..... 53
Equivalent while Loop..... 53
3) do-while Loop..... 53
Example 1: Print Numbers from 1 to 10..... 55
Example 2: Sum of First 10 Natural Numbers..... 56
Output..... 58
Different Conditions in do-while..... 58
5. Jump Control Instructions..... 62
(a) break Statement..... 62
(b) continue Statement..... 64
(c) goto Statement..... 65
(d) return Statement..... 67
Output..... 69

Programming Questions on Decision Making Statements....69
Programming Questions on for Loop....71
Programming Questions on while Loop....72
Intermediate....72
Programming Questions on do-while Loop....73
Mixed Practice Questions....73
Pattern Programming Questions in C....73

Introduction

Control instructions are statements that **control the flow of execution** of a program.

They decide:

- Which statement executes
- How many times it executes
- In what order it executes

Types of Control Instructions

Decision Control Instructions in C

1. **if Statement**
2. **if-else Statement**
3. **Nested if-else Statement**
4. **else-if Ladder**
5. **switch Statement**
6. **Conditional (Ternary) Operator ?:**

Loop Control Instructions in C

1. **while Loop**
2. **do-while Loop**
3. **for Loop**
4. **Nested for Loop**
5. **Nested while Loop**
6. **Nested do-while Loop**

Jump Control Instructions in C

1. **break**
2. **continue**
3. **goto**
4. **return**

Decision Control Instructions

Used to **make decisions based on conditions**

(a) if Statement

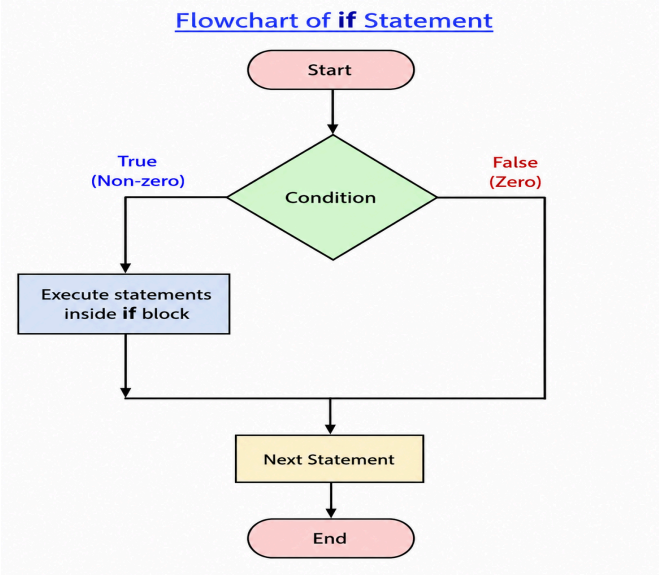
The **if statement** is a decision control statement used to execute a block of code **only when a condition is true**.

If the condition is false, the statements inside the if block are skipped.

Syntax :

```
if(condition){
statements;
}
```

Flowchart:



How if Works

Step 1

The condition inside if() is evaluated.

if(condition)

Step 2

If the condition is **true (non-zero)**:

{

  statements;

}

are executed.

Step 3

If the condition is **false (zero)**: The statements inside the block are skipped.

Step 4

The program continues with the next statement after the if block.

Example 1: Simple if

```c
#include <stdio.h>

int main()
{
   int a = 10;

   if(a > 5)
   {
      printf("Condition is True");
   }

   printf("\nProgram End");

   return 0;
}
```

Step-by-Step Execution

Step 1

a = 10

Step 2

Condition checked: a > 5

Substitute value: 10 > 5

Result: True

Step 3

Execute: printf("Condition is True");

Step 4

Execute next statement: printf("Program End");

Output

Condition is True

Program End

Example 2: Condition is False

```c
#include <stdio.h>

int main()
{
   int a = 2;

   if(a > 5)
   {
      printf("Condition is True");
   }

   printf("Program End");

   return 0;
}
```

Step-by-Step Execution

Step 1 : a = 2

Step 2

Condition checked: 2 > 5

Result: False

Step 3

Skip: printf("Condition is True");

Step 4

Execute: printf("Program End");

Conditions Used in if()

The condition inside if() can be many different types.

1. Relational Expression

Uses relational operators.

Example: if(a > b)

Possible operators: >, <, >= , <= , == , !=

Example Program

```
#include <stdio.h>

int main()
{
   int a = 20;
   int b = 10;

   if(a > b)
   {
      printf("a is greater");
   }

   return 0;
}
```

Output: a is greater

2. Variable as Condition

A variable can be used directly.

Example

```
int a = 10;
if(a)
{
   printf("True");
}
```

Output: True

Because: a = 10

Non-zero value means true.

Example

int a = 0;

```
if(a)
{
  printf("True");
}
```

Output: No Output

Because: 0 = False

3. Constant as Condition

Example

```
if(1)
{
  printf("Hello");
}
```

Output: Hello

Example

```
if(100)
{
   printf("Hello");
}
```

Output: Hello

Because: 100 ≠ 0 , So it is true.

Example

```
if(0)
```

```
{
   printf("Hello");
}
```

Output: No Output

Because: 0 = False

4. Arithmetic Expression

Example

```
if(5 + 3)
{
   printf("True");
}
```

Calculation: 5 + 3 = 8

Since: 8 ≠ 0

Output: True

Example

```
if(5 - 5)
{
   printf("True");
}
```

Calculation: 5 - 5 = 0

Output: No Output

5. Logical Expression

Example

```
if(10 > 5 && 20 > 15)
{
   printf("True");
```

```
}
```

Evaluate:

10 > 5 = True

20 > 15 = True

True && True = True

Output: True

Example Program: Check Positive Number

```
#include <stdio.h>

int main()
{
   int num;

   printf("Enter number: ");
   scanf("%d",&num);

   if(num > 0)
   {
      printf("Positive Number");
   }

   return 0;
}
```

Input : 15

Condition : 15 > 0

Result: True

Output: Positive Number

Input : -5

Condition: -5 > 0

Result: False

Output: No Output, because there is no else block.

Single Statement if

Braces are optional when only one statement exists.

```
if(a > b)
   printf("a is greater");
```

Equivalent to:

```
if(a > b)
{
   printf("a is greater");
}
```

Multiple Statements

Braces are mandatory.

```
if(a > b)
{
   printf("a is greater");
   printf("\nCondition True");
}
```

(b) if-else

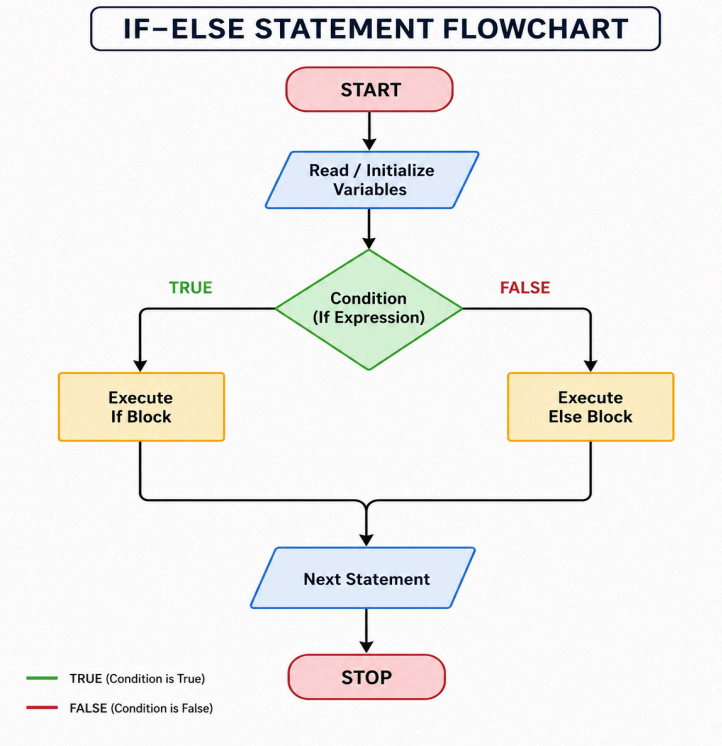
The **if-else statement** is a decision control statement used to execute one block of code when a condition is true and another block when the condition is false.

It allows the program to choose between **two alternatives**.

Syntax

```
if(condition)
{
   // statements executed when condition is true
}
else
{
   // statements executed when condition is false
}
```

Flowchart



How if-else Works

Step 1 : The condition is evaluated.

Step 2 : If the condition is **true (non-zero)**: if block is executed.

Step 3 : If the condition is **false (zero)**: else block is executed.

Step 4 : Program continues with the next statement after the if-else.

Conditions Used in if

1. Relational Conditions

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
==	Equal to
!=	Not equal to

Example: if(a > b)

2. Logical Conditions

Operator	Meaning
&&	AND
||	OR
!	NOT

Example: if(a > 10 && b < 20)

3. Constant Conditions

Always True : if(1) Or if(100)
Always False : if(0)

4. Arithmetic Expression

if(a+b)
If result is non-zero → True

If result is zero → False

Example:

int a=5,b=3;
if(a+b)

Result: 8 → True

5. Variable as Condition

if(a)
If → a = 0 → False
If → a = 10 → True

Program 1: Check Even or Odd Number

```c
#include <stdio.h>

int main()
{
   int num;

   printf("Enter a number: ");
   scanf("%d",&num);

   if(num % 2 == 0)
   {
       printf("Even Number");
   }
   else
   {
       printf("Odd Number");
   }

  return 0;
}
```

Example Input : 8

Step-by-Step Execution
Step 1
User enters: 8

So: num = 8

Step 2
Condition checked: num % 2 == 0
Substitute value: 8 % 2 == 0
Result: 0 == 0
Result: True

Step 3
Since condition is true: printf("Even Number"); executes.
Output : Even Number

Example Input : 7
Condition : 7 % 2 == 0
Result: 1 == 0 → False
Output : Odd Number

Program 2: Find Greater Number

```c
#include <stdio.h>

int main()
{
   int a,b;

   printf("Enter two numbers: ");
   scanf("%d%d",&a,&b);

   if(a>b)
   {
       printf("%d is Greater",a);
   }
   else
   {
       printf("%d is Greater",b);
   }

   return 0;
}
```

Example Input
20 10

Step-by-Step Execution
Step 1
Input:
a = 20
b = 10

Step 2
Condition checked: a > b
Substitute values: 20 > 10
Result:True

Step 3
Since condition is true:
printf("%d is Greater",a);
executes.

Output : 20 is Greater

Example Input : 5 12
Condition : 5 > 12
Result: False

Step 3
Else block executes: printf("%d is Greater",b);
Output : 12 is Greater

Important Examples of Conditions

Example 1
if(5)
Output: True, Because non-zero means true.

Example 2
if(0)
Output: False, because zero means false.

Example 3
int a=10;
if(a)
Output: True

Example 4
int a=0;
if(a)
Output: False

Example 5
if(10>5)
Output: True

Example 6
if(10<5)
Output: False
Important Notes

1. if executes when condition is true.
2. else executes when condition is false.
3. Condition can be:
    - Relational expression
    - Logical expression
    - Variable
    - Constant
    - Arithmetic expression
4. In C:
    - 0 → False
    - Non-zero → True
5. Only one block (if or else) executes at a time.

(c) else-if ladder

The **else-if ladder** is used when there are **multiple conditions** to check.

The program checks conditions **one by one from top to bottom**. As soon as a condition becomes true, its block executes and the remaining conditions are skipped.

Syntax

```
if(condition1){

  statements1;

}

else if(condition2){
```

```
  statements2;
}
else if(condition3){
  statements3;
}
else{
  statements4;
}
```

How It Works

Step 1

Check condition1.

- If True → Execute Block 1 and exit ladder.
- If False → Check condition2.

Step 2

Check condition2.

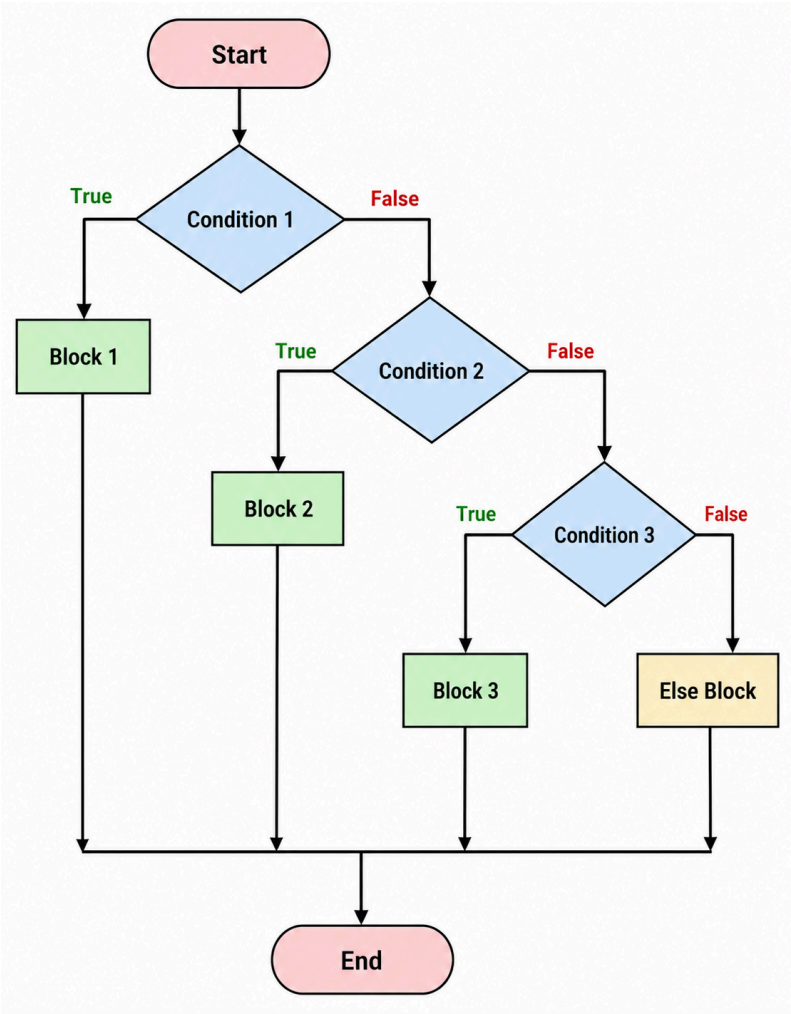
- If True → Execute Block 2 and exit ladder.
- If False → Check condition3.

Step 3

Check condition3.

- If True → Execute Block 3 and exit ladder.
- If False → Execute else block.

Flowchart



Program 1: Find the Largest of Three Numbers

```
#include <stdio.h>

int main()
{
  int a,b,c;

  printf("Enter three numbers: ");
  scanf("%d%d%d",&a,&b,&c);

  if(a>b && a>c)
  {
     printf("A is largest");
```

```
  }
  else if(b>a && b>c)
  {
     printf("B is largest");
  }
  else
  {
     printf("C is largest");
  }

  return 0;
}
```

Example

Input : 10 20 15

Step-by-Step Execution

Step 1

a = 10

b = 20

c = 15

Step 2

Check: a>b && a>c —> 10>20 && 10>15 → False

Step 3

Check: b>a && b>c → 20>10 && 20>15 True

Step 4

Execute: printf("B is largest");

Output

B is largest

Program 2: Grade Calculation

```c
#include <stdio.h>

int main()
{
   int marks;

   printf("Enter Marks: ");
   scanf("%d",&marks);

   if(marks >= 75)
   {
      printf("Distinction");
   }
   else if(marks >= 60)
   {
      printf("First Class");
   }
   else if(marks >= 50)
   {
      printf("Second Class");
   }
   else if(marks >= 35)
   {
      printf("Pass");
   }
   else
   {
      printf("Fail");
   }

   return 0;
}
```

Important Notes

Only One Block Executes

if(condition1)

```
{

}

else if(condition2)

{

}

else if(condition3)

{

}
```

Even if multiple conditions are true, only the **first true condition** executes.

Order of Conditions Matters

Wrong:

```
if(marks >= 35)

{

   printf("Pass");

}

else if(marks >= 75)

{

   printf("Distinction");

}
```

For marks = 80

Output:

Pass

because the first condition becomes true.

Correct Order

```
if(marks >= 75)
{
   printf("Distinction");
}
else if(marks >= 60)
{
   printf("First Class");
}
else if(marks >= 35)
{
   printf("Pass");
}
```

Difference Between if and else-if

if	else-if
Checks one condition	Checks multiple conditions
Independent decision	Part of a ladder
May execute multiple separate if statements	Only one block executes

(d) Nested if-else in C

Definition

A **Nested if-else** means placing one if-else statement inside another if or else block.

It is used when a decision depends on another decision.

In simple words:

> First check one condition. If it is true (or false), then check another condition inside it.

Syntax

```
if(condition1)
{
  if(condition2)
  {
     statements1;
  }
  else
  {
     statements2;
  }
}
else
{
  statements3;
}
```

How It Works

Step 1

Check condition1

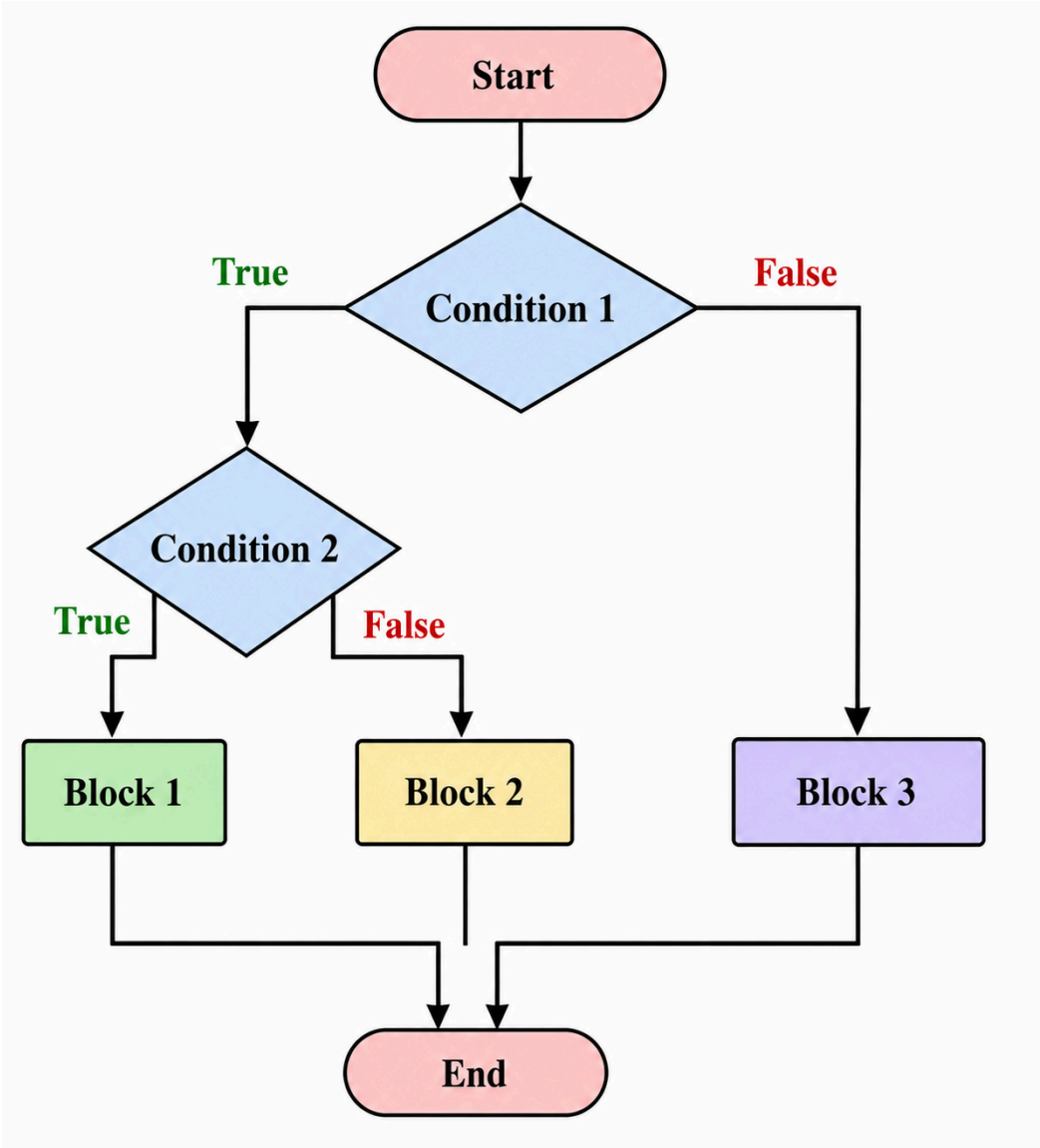
- If True → Go inside the first if block.
- If False → Execute the outer else block.

Step 2

If condition1 is True, check condition2.

- If True → Execute statements1.
- If False → Execute statements2.

Flowchart



Program 1: Check Positive and Even Number

```c
#include <stdio.h>

int main()
{
   int num;

   printf("Enter a number: ");
   scanf("%d",&num);
```

```
    if(num > 0)
    {
        if(num % 2 == 0)
        {
            printf("Positive Even Number");
        }
        else
        {
            printf("Positive Odd Number");
        }
    }
    else
    {
        printf("Number is Negative or Zero");
    }

    return 0;
}
```

Program 2: Find Largest of Three Numbers

```
#include <stdio.h>

int main()
{
    int a,b,c;

    printf("Enter three numbers: ");
    scanf("%d%d%d",&a,&b,&c);

    if(a>b)
    {
        if(a>c)
        {
            printf("A is Largest");
        }
        else
        {
            printf("C is Largest");
```

```
    }
  }
  else
  {
    if(b>c)
    {
      printf("B is Largest");
    }
    else
    {
      printf("C is Largest");
    }
  }

  return 0;
}
```

Difference Between if-else and Nested if-else

if-else	Nested if-else
Checks one condition	Checks a condition inside another condition
Two possible paths	Multiple dependent decisions
Simpler structure	More complex structure

(e) switch Statement

Syntax:

```
switch (expression)
 {
    case constant1:
       // statements
       break;

    case constant2:
       // statements
       break;

    case constant3:
       // statements
       break;

    default:
       // statements (optional)
}
```

Used for multiple choices

switch Statement Rules

1. Switch Expression Rules

- The expression in switch(expression) is evaluated **only once**
- It must be of type:
    - int
    - char
    - enum
- Not allowed:
    - float, double, string
- switch works using a **jump table**, so execution is faster for multiple cases
- If no case matches:
    - Control moves **outside switch** (if no default is present)

2. Case Rules

- Each case represents a **possible value** of the expression
- Cases can be written in **any order** (not necessarily ascending)

- Each case must be **unique**
- Multiple cases can share the **same code.**

```
case 'a':

case 'A':

  printf("Letter A");

  break;
```

- If break is missing:
  - **Fall-through occurs**
  - Execution continues to next case
- No need to use { } for multiple statements in a case
- Every statement inside switch must belong to a **case or default**
  - Otherwise, it will **never execute**

3. Constant Rules (Case Values)

- Case values must be:
  - **Constant values only**
  - Integer constants (1, 2, 10)
  - Character constants ('a', 'x')
- Characters are internally treated as **ASCII values**
  - 'a' = 97, 'x' = 120
- Constant expressions are allowed:

```
case 1+2:   // valid (evaluates to 3)
```

Not Allowed in case:

- Variables

```
case x:   // invalid
```

- Relational expressions

```
case i < 10:   // invalid
```

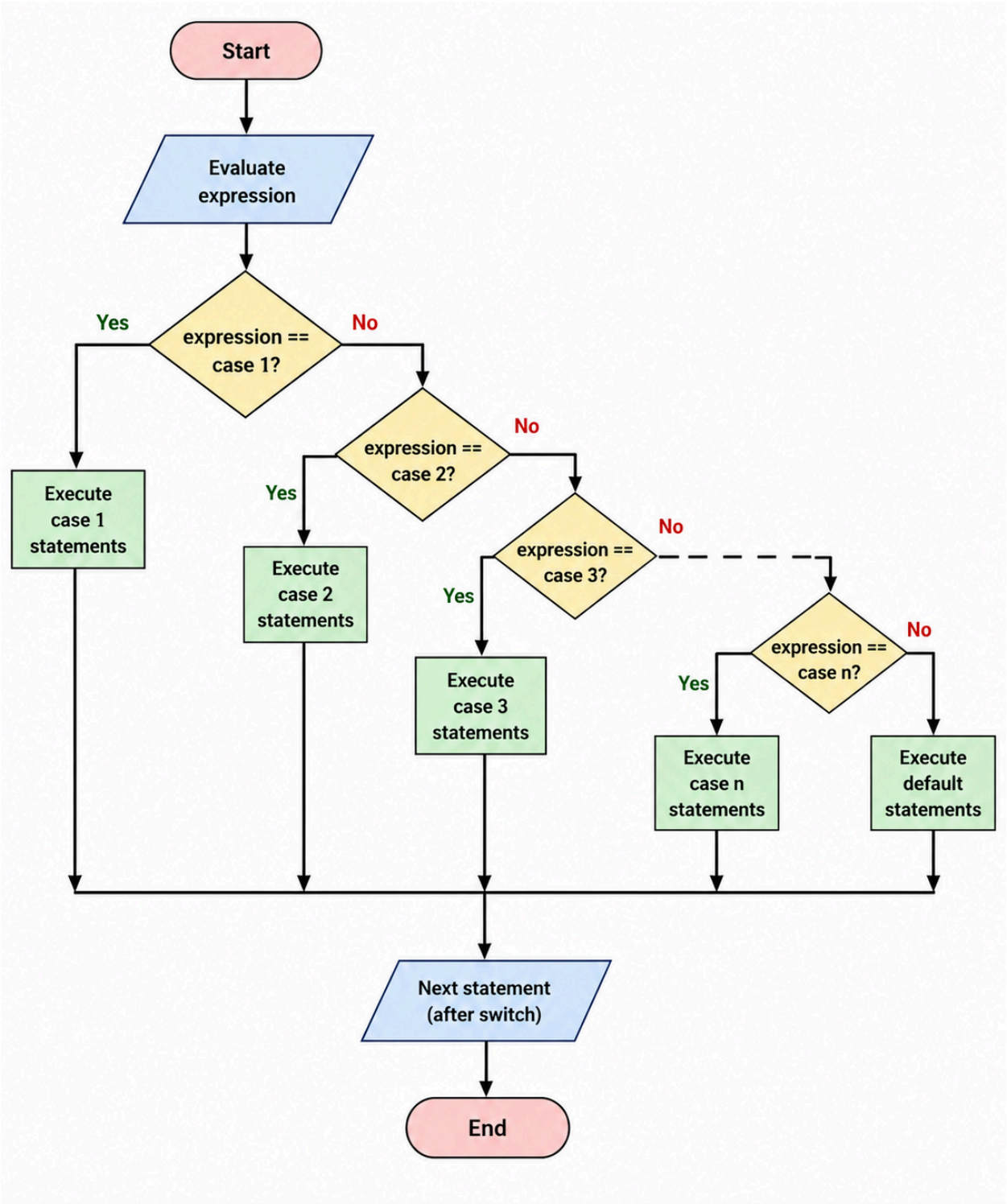
- Floating values

```
case 2.5:   // invalid
```

Why switch is faster than if-else

- switch statement is generally **faster than an equivalent if-else ladder**
- This is because the compiler creates a **jump table** during compilation
- During execution:
    - switch directly **refers to the jump table**
    - It jumps to the required case **without checking each condition**
- In contrast, if-else:
    - Conditions are evaluated **one by one at runtime**
    - This makes it **slower**

Flow Chart


Start
Evaluate expression
expression == case 1?
Yes
No
Execute case 1 statements
expression == case 2?
Yes
No
Execute case 2 statements
expression == case 3?
Yes
No
Execute case 3 statements
expression == case n?
Yes
No
Execute case n statements
Execute default statements
Next statement (after switch)
End

Examples of switch-case in C

Example 1: Display Day of Week

```c
#include <stdio.h>

int main()
{
  int day;

  printf("Enter day number (1-7): ");
  scanf("%d", &day);

  switch(day)
  {
     case 1:
        printf("Monday");
        break;

     case 2:
        printf("Tuesday");
        break;

     case 3:
        printf("Wednesday");
        break;

     case 4:
        printf("Thursday");
        break;

     case 5:
        printf("Friday");
        break;

     case 6:
        printf("Saturday");
        break;

     case 7:
        printf("Sunday");
        break;
```

```
    default:
        printf("Invalid Day");
  }

  return 0;
}
```

Example 2: Simple Calculator

```
#include <stdio.h>

int main()
{
  int a, b;
  char op;

  printf("Enter two numbers: ");
  scanf("%d%d", &a, &b);

  printf("Enter operator (+,-,*,/): ");
  scanf(" %c", &op);

  switch(op)
  {
    case '+':
        printf("Result = %d", a + b);
        break;

    case '-':
        printf("Result = %d", a - b);
        break;

    case '*':
        printf("Result = %d", a * b);
        break;

    case '/':
        printf("Result = %d", a / b);
        break;
```

```
    default:
        printf("Invalid Operator");
  }

  return 0;
}
```

Example 3: Check Vowel or Consonant

```
#include <stdio.h>

int main()
{
  char ch;

  printf("Enter a character: ");
  scanf("%c", &ch);

  switch(ch)
  {
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u':
    case 'A':
    case 'E':
    case 'I':
    case 'O':
    case 'U':
        printf("Vowel");
        break;

    default:
        printf("Consonant");
  }

  return 0;
}
```

Example 4: Menu Driven Program

```c
#include <stdio.h>

int main()
{
  int choice;

  printf("\n1. Add");
  printf("\n2. Subtract");
  printf("\n3. Multiply");
  printf("\n4. Exit");

  printf("\nEnter your choice: ");
  scanf("%d", &choice);

  switch(choice)
  {
     case 1:
        printf("Addition Selected");
        break;

     case 2:
        printf("Subtraction Selected");
        break;

     case 3:
        printf("Multiplication Selected");
        break;

     case 4:
        printf("Program Terminated");
        break;

     default:
        printf("Invalid Choice");
  }

  return 0;
}
```

Example 5: Even or Odd Using switch

```c
#include <stdio.h>

int main()
{
  int num;

  printf("Enter a number: ");
  scanf("%d",&num);

  switch(num % 2)
  {
     case 0:
        printf("Even Number");
        break;

     case 1:
        printf("Odd Number");
        break;
  }

  return 0;
}
```

Example 6: Arithmetic Operations

```c
#include <stdio.h>

int main()
{
  int a = 20;
  int b = 10;
  int choice;

  printf("1.Add\n");
  printf("2.Subtract\n");
  printf("3.Multiply\n");
  printf("4.Divide\n");
```

```
    printf("Enter Choice: ");
    scanf("%d",&choice);

    switch(choice)
    {
        case 1:
            printf("Sum = %d", a+b);
            break;

        case 2:
            printf("Difference = %d", a-b);
            break;

        case 3:
            printf("Product = %d", a*b);
            break;

        case 4:
            printf("Quotient = %d", a/b);
            break;

        default:
            printf("Invalid Choice");
    }

    return 0;
}
```

Loop Control Instructions

Used to **repeat a block of code.**

1) for Loop in C

A **for loop** is a loop control statement used to execute a block of code repeatedly for a specified number of times.

It is generally used when the number of iterations is known in advance.

Syntax

```
for(initialization; condition; increment/decrement)

{

   statements;

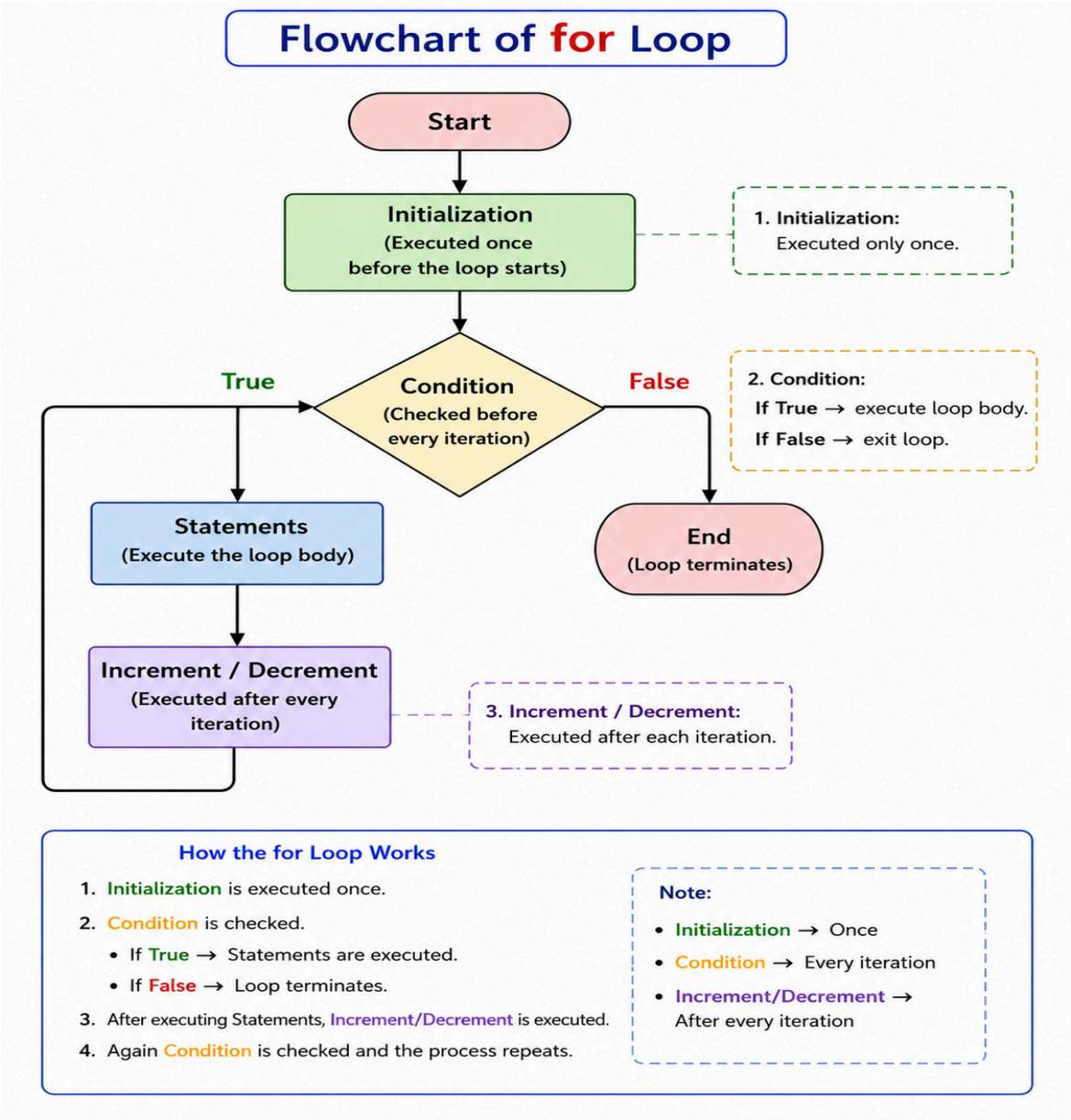
}
```

Structure of for Loop

```
for(i = 1; i <= 5; i++)
{
   printf("%d ", i);
}
```

The for loop consists of **three parts**:

Part	Purpose
Initialization	Executes only once before the loop starts
Condition	Checked before every iteration
Increment/Decrement	Executes after every iteration



Working of for Loop

Example:

```
for(i=1; i<=3; i++)
{
   printf("%d ",i);
}
```

Step 1

Initialization executes. → i = 1;

Step 2

Condition checked.

i <= 3 → True

Step 3

Execute loop body.

printf("%d ",i);

Output: 1

Step 4

Increment executes. i++, now i = 2;

Step 5

Condition checked again.

2 <= 3 → True

Repeat until condition becomes false.

Output:

1 2 3

Different Conditions in for Loop

1. Incrementing Loop

for(i=1; i<=5; i++)

Output:

1 2 3 4 5

2. Decrementing Loop

for(i=5; i>=1; i--)

Output:

5 4 3 2 1

3. Step Size Greater Than 1

for(i=0; i<=10; i=i+2)

Output:

0 2 4 6 8 10

4. Multiple Variables

```
for(i=1,j=10; i<=5; i++,j--)
```

5. Infinite Loop

```
for(;;)
{
   printf("Hello");
}
```

Condition is omitted. The loop runs forever.

---

6. Missing Initialization

```
int i=1;
```

```
for(; i<=5; i++)
{
   printf("%d ",i);
}
```

Valid.

7. Missing Increment

```
for(i=1; i<=5;)
{
   i++;
}
```

Valid.

8. Missing Condition

```
for(i=1;;i++)
{
}
```

Infinite loop.

9. Empty for Loop

```
for(;;)
{
}
```

Infinite loop.

Example 1: Print Numbers from 1 to 10

```
#include <stdio.h>

int main()
{
  int i;

  for(i=1; i<=10; i++)
  {
     printf("%d\n",i);
  }

  return 0;
}
```

Step-by-Step Execution

Iteration 1

i=1 → Check: 1<=10 → True → Print: 1 → Increment: i=2

Iteration 2

2<=10 → True → Print: 2 → Increment: i=3

This continues until:

i=11 → Condition: 11<=10 → False → Loop terminates.

Output

1

2

3

4

5

6

7

8

9

10

Example 2: Find Sum of First 10 Natural Numbers

```
#include <stdio.h>

int main()
{
   int i, sum=0;
```

```
   for(i=1; i<=10; i++)
   {
      sum = sum + i;
   }

   printf("Sum = %d",sum);

   return 0;
}
```

Step-by-Step Execution

Initial: sum = 0

Iteration 1 : i=1 → sum = 0 + 1 → sum = 1

Iteration 2 : i=2 → sum = 1 + 2 → sum = 3

Iteration 3 : i=3 → sum = 3 + 3 → sum = 6

Continue:

i	sum
1	1
2	3
3	6
4	10
5	15

6	21
7	28
8	36
9	45
10	55

Output

Sum = 55

Nested for Loop

A for loop inside another for loop.

```
for(i=1; i<=3; i++)
{
   for(j=1; j<=3; j++)
   {
      printf("* ");
   }

   printf("\n");
}
```

Output:

* * *

* * *

* * *

2) while Loop

A **while loop** is a loop control statement that repeatedly executes a block of code **as long as a specified condition is true**.

The condition is checked **before** executing the loop body. Therefore, a while loop is called an **entry-controlled loop**.

Syntax

```
while(condition)

{

   statements;

}
```

General Form

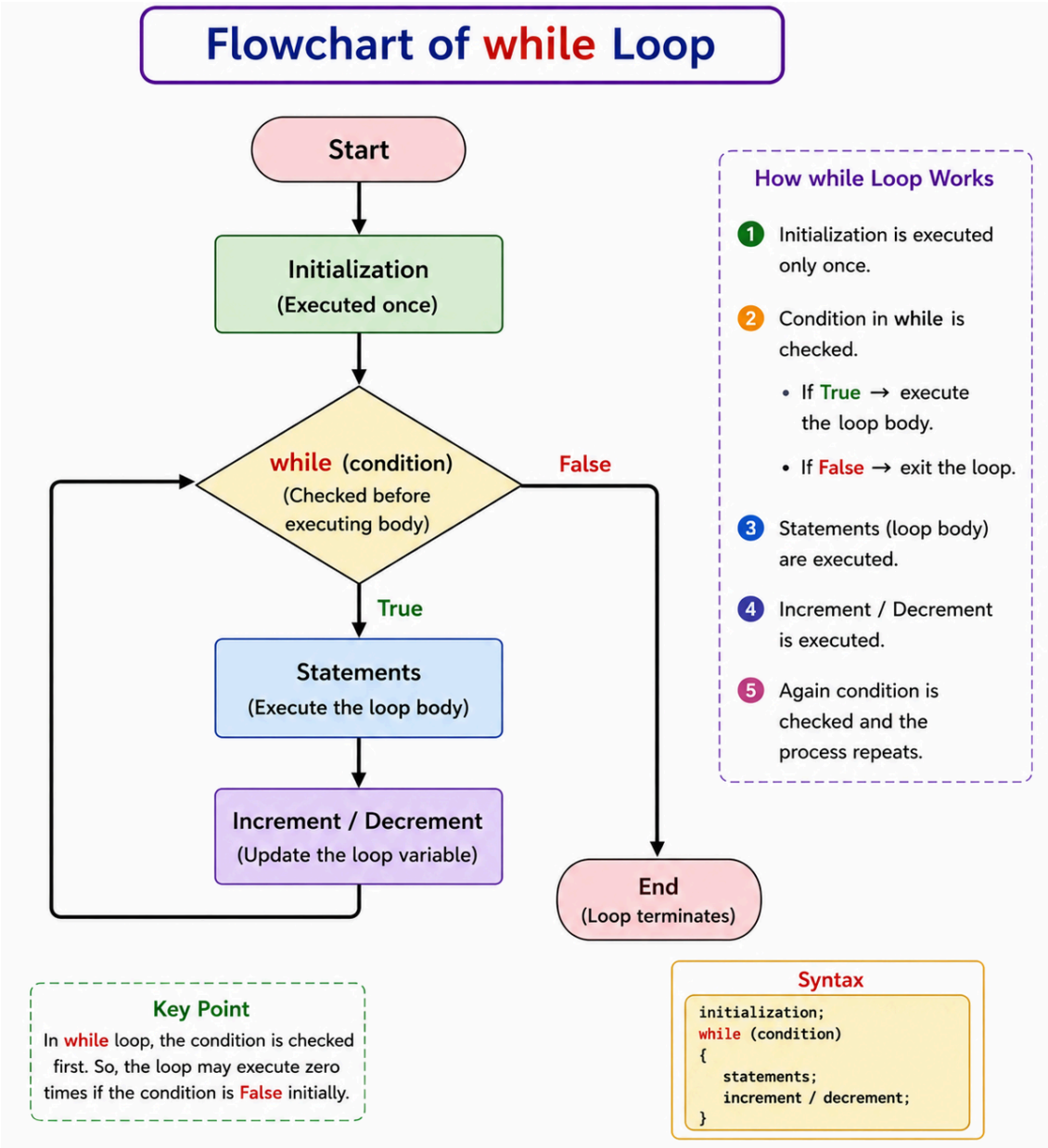
```
initialization;

while(condition)

{
  statements;

   increment/decrement;

}
```



How while Loop Works

Example:

```
int i = 1;
while(i <= 3)
{
   printf("%d ", i);
```

```
    i++;
}
```

Step 1
Initialization: i = 1;

Step 2
Condition checked: i <= 3 → 1 <= 3 → True

Step 3
Execute → printf("%d ", i); → **Output: 1**

Step 4
Increment: i++; Now: i = 2;

Step 5
Condition checked again. → 2 <= 3 → True

Repeat.

Final Output

1 2 3

Example 1: Print Numbers from 1 to 10

Program

```
#include <stdio.h>

int main()
{
    int i = 1;

    while(i <= 10)
    {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

Example 2: Sum of First 10 Natural Numbers

Program

```c
#include <stdio.h>

int main()
{
  int i = 1;
  int sum = 0;

  while(i <= 10)
  {
     sum = sum + i;
     i++;
  }

  printf("Sum = %d", sum);

  return 0;
}
```

Different Conditions in while Loop

1. Incrementing Loop

```c
int i = 1;
while(i <= 5)
{
   printf("%d ", i);
   i++;
}
```

Output:

1 2 3 4 5

2. Decrementing Loop

```c
int i = 5;
while(i >= 1)
{
   printf("%d ", i);
   i--;
}
```

Output:

5 4 3 2 1

3. Infinite Loop

```c
while(1)
{
   printf("Hello");
}
```

Condition is always true. The loop never terminates.

4. Variable as Condition

```c
int a = 10;
while(a)
{
   printf("%d ", a);
   a--;
}
```

Output:

10 9 8 7 6 5 4 3 2 1

When: a = 0; , Condition becomes false.

5. Logical Condition

```c
while(i <= 10 && j <= 20)
{
   statements;
}
```

Loop continues only if both conditions are true.

Nested while Loop

A while loop inside another while loop.

```c
#include <stdio.h>

int main()
{
   int i = 1, j;

   while(i <= 3)
   {
      j = 1;

      while(j <= 3)
      {
         printf("* ");
         j++;
      }

      printf("\n");
      i++;
   }

   return 0;
}
```

Output:

* * *

* * *

* * *

Difference Between for and while

for Loop	while Loop

Initialization, condition and increment are in one place	Written separately
Used when number of iterations is known	Used when number of iterations is not known
Compact syntax	Simple and flexible

for Loop

```
for(i=1; i<=10; i++)
{
   printf("%d", i);
}
```

Equivalent while Loop

```
i = 1;
while(i <= 10)
{
   printf("%d", i);
   i++;
}
```

---

3) do-while Loop

A **do-while loop** is a loop control statement that executes a block of code **at least once** and then repeatedly executes it as long as the specified condition is true.The condition is checked **after** executing the loop body. Therefore, a do-while loop is called an **exit-controlled loop**.

Syntax

```
do
{
   statements;
```

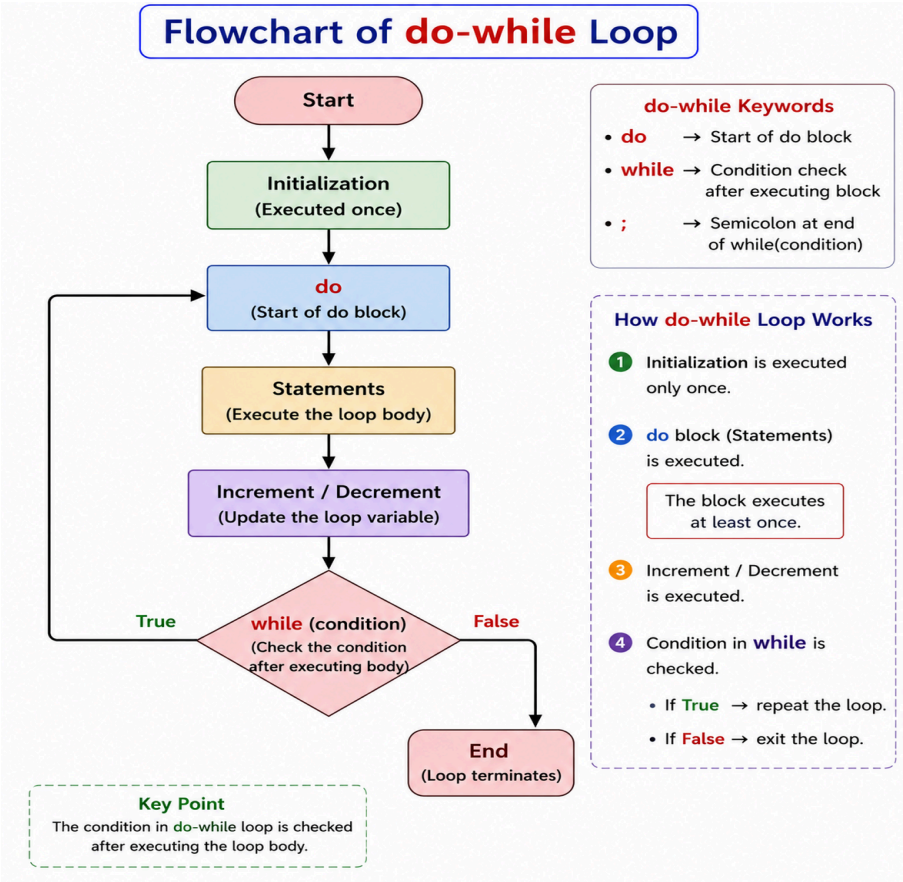
```
}
while(condition);
```

General Form

```
initialization;

do
{
  statements;

  increment/decrement;
}
while(condition);
```



How do-while Works

Example:

```
int i = 1;

do
{
   printf("%d ", i);
   i++;
}
while(i <= 3);
```

Step 1

Initialization: i = 1;

Step 2

Execute loop body.

Output: 1

Step 3

Increment: i = 2;

Step 4

Check condition: 2 <= 3 → True

Loop repeats.

Final Output
1 2 3

Example 1: Print Numbers from 1 to 10

```
#include <stdio.h>

int main()
{
   int i = 1;
```

```
    do
    {
        printf("%d\n", i);
        i++;
    }
    while(i <= 10);

    return 0;
}
```

Step-by-Step Execution

Initial Value → i = 1;

Iteration 1: Print: 1 → **Increment:** i = 2; → **Check:** 2 <= 10 → True

Iteration 2: Print: 2 → **Increment:** i = 3; → **Check:** 3 <= 10 → True

This process continues until: i = 11; → Condition: 11 <= 10 → False

Loop terminates.

Output

1
2
3
4
5
6
7
8
9
10

Example 2: Sum of First 10 Natural Numbers

Program

```
#include <stdio.h>

int main()
{
```

```c
    int i = 1;
    int sum = 0;

    do
    {
        sum = sum + i;
        i++;
    }
    while(i <= 10);

    printf("Sum = %d", sum);

    return 0;
}
```

Step-by-Step Execution

Iteration	i	sum
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15
6	6	21
7	7	28
8	8	36
9	9	45
10	10	55

Output : Sum = 55

Important Feature of do-while

The loop body executes **at least once**, even if the condition is false.

Example

```c
#include <stdio.h>

int main()
{
   int i = 10;

   do
   {
      printf("%d", i);
   }
   while(i < 5);

   return 0;
}
```

Execution

Print: 10 → Check: 10 < 5 → False

Loop terminates.

Output

10

Even though the condition is false, the loop executes once.

Different Conditions in do-while

1. Incrementing Loop

```c
int i = 1;

do
{
   printf("%d ", i);
   i++;
}
while(i <= 5);
```

Output:

1 2 3 4 5

2. Decrementing Loop

```
int i = 5;

do
{
   printf("%d ", i);
   i--;
}
while(i >= 1);
```

Output:

5 4 3 2 1

3. Infinite Loop

```
do
{
   printf("Hello");
}
while(1);
```

Condition is always true. Loop never terminates.

4. Variable as Condition

```
int a = 5;

do
{
   printf("%d ", a);
   a--;
}
while(a);
```

Output: 5 4 3 2 1

5. Logical Condition

```
do
{
  statements;
}
while(i <= 10 && j <= 20);
```

Loop continues only if both conditions are true.

Nested do-while Loop

A do-while loop inside another do-while loop.

```
#include <stdio.h>

int main()
{
  int i = 1, j;

  do
  {
     j = 1;

     do
     {
        printf("* ");
        j++;
     }
     while(j <= 3);

     printf("\n");
     i++;
  }
  while(i <= 3);

  return 0;
}
```

Output

```
* * *
* * *
```

* * *

Example

while Loop

```
int i = 10;

while(i < 5)
{
   printf("Hello");
}
```

Output:

No Output

do-while Loop

```
int i = 10;

do
{
   printf("Hello");
}
while(i < 5);
```

Output:

Hello

Difference Between for, while, and do-while

Feature	for	while	do-while
Control Type	Entry	Entry	Exit
Condition Check	Before	Before	After
Executes At Least Once	No	No	Yes

Best Used When	Iterations known	Iterations unknown	At least one execution required

Difference: while vs do-while

Feature	while Loop	do-while Loop
Control Type	Entry-controlled loop	Exit-controlled loop
Condition Check	Before executing loop body	After executing loop body
Minimum Execution	May execute **0 times**	Executes **at least once**
Syntax Ending	No semicolon at end	Ends with **semicolon (;)**
Use Case	When condition may be false initially	When code must run at least once

Jump Control Instructions

Jump control statements change the normal flow of program execution.

They are:

1. break
2. continue
3. goto
4. return

(a) break Statement

The break statement is used to **immediately terminate a loop or switch statement**.

When break executes, control moves to the first statement after the loop or switch.

Syntax:

break;

Example: break in a Loop

```c
#include <stdio.h>

int main()
{
   int i;

   for(i=1; i<=10; i++)
   {
      if(i==6)
      {
         break;
      }

      printf("%d ",i);
   }

   return 0;
}
```

Step-by-Step Execution

Iterations

i	Condition (i==6)	Action
1	False	Print 1
2	False	Print 2
3	False	Print 3
4	False	Print 4
5	False	Print 5
6	True	break executes

Loop terminates immediately.

Output

1 2 3 4 5

(b) continue Statement

The continue statement skips the remaining statements of the current iteration and moves directly to the next iteration of the loop.

Unlike break, it does not terminate the loop.

Syntax

continue;

Example: continue in a Loop

```
#include <stdio.h>

int main()
{
   int i;

   for(i=1; i<=10; i++)
   {
      if(i==5)
      {
         continue;
      }

      printf("%d ",i);
   }

   return 0;
}
```

Step-by-Step Execution

i	Condition (i==5)	Action
1	False	Print 1
2	False	Print 2

3	False	Print 3
4	False	Print 4
5	True	Skip printf
6	False	Print 6
7	False	Print 7
8	False	Print 8
9	False	Print 9
10	False	Print 10

Output

1 2 3 4 6 7 8 9 10

Notice that **5 is skipped**.

Difference Between break and continue

break	continue
Terminates the loop	Skips current iteration
Control exits loop	Control goes to next iteration
Remaining iterations are not executed	Remaining iterations continue

(c) goto Statement

Definition

The goto statement transfers program control directly to a labeled statement.

Syntax

goto label;

/* statements */

label:
statement;

Example

```c
#include <stdio.h>

int main()
{
   int i=1;

start:

   printf("%d ",i);

   i++;

   if(i<=5)
   {
      goto start;
   }

   return 0;
}
```

Step-by-Step Execution

First Time
i=1

Print: 1

Increment: i=2 , Condition: 2<=5 → True

Execute: goto start;

Program jumps back to label: start:

This repeats until: i=6

Condition: 6<=5 → False → Program exits.

Output

1 2 3 4 5

Note:

Although goto is valid, it should be used carefully because excessive use makes programs difficult to understand and debug.

(d) return Statement

Definition

The return statement terminates a function and optionally sends a value back to the calling function.

Syntax

return value;

or

return;

Example

```
#include <stdio.h>

int add(int a,int b)
{
   return a+b;
}

int main()
{
   int result;

   result = add(10,20);

   printf("Sum = %d",result);

   return 0;
```

```
}
```

Step-by-Step Execution

Step 1

Call: add(10,20);

Control moves to: int add(int a,int b)

Step 2

Parameters receive values: a=10 ,b=20

Step 3

Execute: return a+b;

Calculation: return 30;

Step 4

Function terminates and returns 30 to result so result = 30;

Step 5

Print: Sum = 30

Output

Sum = 30

Example of return in main()

```c
#include <stdio.h>

int main()
{
   printf("Program Started");

   return 0;
```

```
    printf("Program End");
}
```

Output

Program Started

Summary Table

Statement	Purpose	Example
break	Exit loop or switch immediately	break;
continue	Skip current iteration	continue;
goto	Jump to labeled statement	goto start;
return	Exit function and return value	return x;

Programming Questions on Decision Making Statements

Simple if

1. Check whether a number is positive.
2. Check whether a number is negative.
3. Check whether a number is divisible by 5.
4. Check whether a number is divisible by both 3 and 7.
5. Check whether a character is an uppercase letter.
6. Check whether a character is a lowercase letter.
7. Check whether a person is eligible to vote.
8. Check whether a number is even.
9. Check whether a year is a leap year.
10. Check whether a student has passed (marks ≥ 35).

---

if-else

1. Check whether a number is even or odd.
2. Find the greater of two numbers.
3. Find the smaller of two numbers.
4. Check whether a character is a vowel or consonant.
5. Check whether a number is positive or negative.
6. Check whether a year is leap year or not.
7. Check whether a number is divisible by 5 and 11.
8. Check whether a person can vote or not.
9. Check whether a character is alphabet or not.
10. Check whether a number is a multiple of 10.

Else-if Ladder

1. Find the largest among three numbers.
2. Calculate grade based on marks.
3. Display division (Distinction, First Class, Second Class, Pass, Fail).
4. Determine profit or loss.
5. Find the greatest among four numbers.
6. Display month name from month number.
7. Display day name from day number.
8. Calculate electricity bill based on units consumed.
9. Determine income tax slab.
10. Find roots category based on discriminant value.

Nested if-else

1. Check whether a number is positive and even.
2. Find the largest among three numbers.
3. Determine admission eligibility based on marks and age.
4. Check whether a year is leap year and century year.
5. Find the second largest among three numbers.
6. Determine scholarship eligibility.
7. Check whether a triangle is valid and identify its type.
8. Find the greatest among four numbers using nested if.
9. Check employee bonus eligibility.
10. Validate username and password.

---

switch-case

1. Display day name using day number.
2. Display month name using month number.
3. Simple calculator using switch.
4. Check vowel or consonant.
5. Menu-driven arithmetic operations.
6. Print number in words (0–9).
7. ATM menu program.
8. Display grade description.
9. Unit conversion menu.
10. Area calculation menu.

Programming Questions on for Loop

Basic

1. Print numbers from 1 to 10.
2. Print numbers from 10 to 1.
3. Print even numbers from 1 to 100.
4. Print odd numbers from 1 to 100.
5. Print multiplication table of a number.
6. Find sum of first N natural numbers.
7. Find sum of even numbers up to N.
8. Find sum of odd numbers up to N.
9. Find factorial of a number.
10. Find square of numbers from 1 to N.

Intermediate

11. Reverse a number.
12. Count digits in a number.
13. Find sum of digits.
14. Check palindrome number.
15. Check Armstrong number.
16. Check prime number.
17. Print all prime numbers from 1 to N.
18. Find HCF of two numbers.
19. Find LCM of two numbers.
20. Generate Fibonacci series.

Pattern Programs

21. Print right triangle star pattern.
22. Print inverted triangle pattern.
23. Print pyramid pattern.
24. Print Floyd's triangle.
25. Print Pascal's triangle.
26. Print diamond pattern.
27. Print number triangle pattern.
28. Print alphabet triangle pattern.
29. Print hollow square pattern.
30. Print hollow pyramid pattern.

Programming Questions on while Loop

Basic

1. Print numbers from 1 to N.
2. Print numbers from N to 1.
3. Print even numbers up to N.
4. Print odd numbers up to N.
5. Find sum of first N natural numbers.
6. Find factorial of a number.
7. Print multiplication table.
8. Reverse a number.
9. Count digits in a number.
10. Find sum of digits.

Intermediate

11. Check palindrome number.
12. Check Armstrong number.
13. Generate Fibonacci series.
14. Check prime number.
15. Print prime numbers between two numbers.
16. Find HCF.
17. Find LCM.
18. Convert decimal to binary.
19. Convert binary to decimal.
20. Find product of digits.

Programming Questions on do-while Loop

Basic

1. Print numbers from 1 to N.
2. Print numbers from N to 1.
3. Print even numbers up to N.
4. Print odd numbers up to N.
5. Find sum of first N natural numbers.
6. Find factorial of a number.
7. Reverse a number.
8. Count digits in a number.
9. Find sum of digits.
10. Generate Fibonacci series.

Mixed Practice Questions

1. Check whether a number is positive, negative, or zero.
2. Find the largest among three numbers.
3. Find the smallest among three numbers.
4. Check whether a number is prime.
5. Check whether a number is Armstrong.
6. Check whether a number is palindrome.
7. Generate Fibonacci series.
8. Find factorial of a number.
9. Calculate power of a number.
10. Find HCF and LCM.
11. Print all prime numbers between 1 and 100.
12. Print multiplication tables from 1 to 10.
13. Display grades using switch-case.
14. Create a menu-driven calculator.
15. Check whether a year is leap year.
16. Count positive, negative, and zero values from a set of inputs.
17. Find the largest digit in a number.
18. Find the smallest digit in a number.
19. Check whether a number is a perfect number.

Pattern Programming Questions in C

Level 1: Basic Star Patterns

1. Print 5 stars in a single line

*****

---

2. Print a square pattern

*****

*****

*****

*****

*****

---

3. Print a rectangular pattern

******

******

******

******

---

4. Print a right-angled triangle

*

**

***

****

*****

5. Print an inverted right triangle

```
*****
****
***
**
*
```

---

6. Print a left-aligned inverted triangle

```
*****
****
 ***
  **
   *
```

---

7. Print a half pyramid

```
*
**
***
****
*****
******
```

---

8. Print an inverted half pyramid

```
******
*****
****
***
**
*
```

Level 2: Full Pyramid Patterns

9. Print a pyramid

```
   *
  ***
 *****
*******
*********
```

---

10. Print an inverted pyramid

```
*********
*******
 *****
  ***
   *
```

---

11. Print a diamond pattern

```
   *
  ***
 *****
*******
*********
*******
 *****
  ***
   *
```

---

12. Print a hollow pyramid

```
   *
  * *
 *   *
*     *
*********
```

---

13. Print a hollow inverted pyramid

```
*********
*     *
 *   *
  * *
```

```
  *
```

---

14. Print a hollow diamond

```
  *
 * *
*   *
*     *
*   *
 * *
  *
```

Level 3: Number Patterns

15. Print numbers from 1 to 5

```
1
12
123
1234
12345
```

---

16. Reverse number triangle

```
12345
```

1234

123

12

1

---

17. Incrementing number pattern

1

23

456

78910

---

18. Floyd's Triangle

1

2 3

4 5 6

7 8 9 10

11 12 13 14 15

---

19. Continuous Number Pyramid

1

2 3

4 5 6

7 8 9 10

---

20. Number Pyramid

```
   1
  121
 12321
1234321
123454321
```

---

21. Palindrome Number Pattern

```
1
22
333
4444
55555
```

---

22. Pascal Triangle

```
      1
    1   1
  1   2   1
1   3   3   1
1   4   6   4   1
```

Level 4: Alphabet Patterns

---

23. Alphabet Triangle

A

AB

ABC

ABCD

ABCDE

---

24. Reverse Alphabet Triangle

ABCDE

ABCD

ABC

AB

A

---

25. Continuous Alphabet Pattern

A

BC

DEF

GHIJ

KLMNO

---

26. Alphabet Pyramid

A

ABA

ABCBA

ABCDCBA

ABCDEDCBA

---

27. Repeated Alphabet Pattern

A

BB

CCC

DDDD

EEEEE

Level 5: Hollow Patterns

28. Hollow Square

*****

*   *

*   *

*   *

*****

---

29. Hollow Rectangle

******

*   *

```
*     *
 ******
```

---

30. Hollow Right Triangle

```
*
**
* *
*  *
*****
```

---

31. Hollow Pyramid

```
    *
  *   *
 *     *
*       *
*********
```

---

32. Hollow Diamond

```
   *
  * *
 *   *
*     *
*       *
```

```
*   *
 *  *
  * *
   *
```

Level 6: Advanced Patterns

33. Butterfly Pattern

```
*     *
**   **
*** ***
********
*** ***
**   **
*     *
```

---

34. X Pattern

```
*     *
*   *
 *  *
  * *
   *
  * *
 *  *
```

```
*     *
*       *
```

---

35. Sandglass Pattern

```
*********
*******
 *****
  ***
   *
  ***
 *****
*******
*********
```